MongoDB Problem Statements

Problem 1: Coffee Shop Menu System

Problem Statement:

A coffee shop "BrewHaven" wants to manage its menu using MongoDB. Each menu item has details such as item_id, name, category (Coffee/Tea/Snack), price, and availability (boolean). The shop owner needs a simple system to manage the menu.

Questions:

1. Create a `menu` collection and insert one menu item document.
2. Insert at least 5 menu items using `insertMany()` with different categories.
3. Retrieve all items with availability true using `find()`.
4. Display only `name` and `price` using projection.
5. Delete a menu item based on item_id.

Problem 2: Contact Book System

Problem Statement:

An individual wants to store personal contacts using MongoDB. Each contact has contact_id, first_name, last_name, phone, email, and city. The user needs a basic system to manage contacts.

Questions:

1. Create a `contacts` collection and insert one contact document.
2. Insert at least 5 contact records using `insertMany()`.
3. Retrieve all contacts from "Mumbai" city using `find()`.
4. Display only `first_name`, `last_name`, and `phone` using projection.
5. Delete a contact based on contact_id.

Problem 3: To-Do List Manager

Problem Statement:

A productivity app wants to manage user tasks using MongoDB. Each task contains task_id, title, description, due_date, priority (Low/Medium/High), and is_completed (boolean). The user needs basic task management.

Questions:

1. Create a tasks collection and insert one task document.
2. Insert at least 5 task records using insertMany() with different priorities.
3. Retrieve all tasks where is_completed is false using find().
4. Display only title and due_date using projection.
5. Delete a task based on task_id.

Problem 4: Expense Tracker System

Problem Statement:

An individual wants to track daily expenses using MongoDB. Each expense contains expense_id, category (Food/Transport/Entertainment/Bills), amount, date, payment_method (Cash/Card/UPI), and note. The user needs a simple system to record and view expenses.

Questions:

1. Create an expenses collection and insert one expense document.
2. Insert at least 5 expense records using insertMany() with different categories.
3. Retrieve all expenses with amount greater than ₹500 using find().
4. Display only category, amount, and date using projection.
5. Delete an expense based on expense_id.

Problem 5: Movie Watchlist System

Problem Statement:

A movie enthusiast wants to maintain a watchlist using MongoDB. Each movie contains movie_id, title, genre, release_year, imdb_rating, and watched (boolean). The user needs a basic system to manage the watchlist.

Questions:

1. Create a watchlist collection and insert one movie document.
2. Insert at least 5 movie records using insertMany() with different genres.
3. Retrieve all movies where watched is false using find().
4. Display only title, genre, and imdb_rating using projection.
5. Delete a movie based on movie_id.

Problem 6: Student Hostel Room Allotment

Problem Statement:

A college hostel wants to manage room allotment using MongoDB. Each student record contains student_id, name, room_number, block_name, bed_number, and check_in_date. The hostel warden needs a basic system to manage allotments.

Questions:

1. Create a hostel collection and insert one student document.
2. Insert at least 5 student records using insertMany() with different room numbers.
3. Retrieve all students from block "A" using find().
4. Display only name, room_number, and block_name using projection.
5. Delete a student record based on student_id.

Problem 7: Parking Lot Management

Problem Statement:

A shopping mall wants to manage its parking lot using MongoDB. Each parking slot contains slot_id, floor, vehicle_type (Car/Bike), is_occupied (boolean), and vehicle_number (if occupied). The security team needs basic operations.

Questions:

1. Create a parking collection and insert one parking slot document.
2. Insert at least 5 parking slot records using insertMany().
3. Retrieve all slots where is_occupied is false using find().
4. Display only slot_id, floor, and vehicle_type using projection.
5. Delete a parking slot based on slot_id.

Problem 8: Gift Card System

Problem Statement:

A retail store wants to manage gift cards using MongoDB. Each gift card contains card_id, recipient_name, sender_name, amount, purchase_date, expiry_date, and is_used (boolean). The store needs basic gift card management.

Questions:

1. Create a giftcards collection and insert one gift card document.
2. Insert at least 5 gift card records using insertMany().
3. Retrieve all cards where is_used is false AND expiry_date is after today's date using find().
4. Display only recipient_name, amount, and expiry_date using projection.
5. Delete a gift card based on card_id.

Problem 9: Courier Parcel Tracker (Basic)

Problem Statement:

A small courier service wants to track parcels using MongoDB. Each parcel contains parcel_id, sender_name, receiver_name, weight, shipping_cost, booking_date, and delivery_status (Pending/Shipped/Delivered). The staff needs basic operations.

Questions:

1. Create a parcels collection and insert one parcel document.

2. Insert at least 5 parcel records using insertMany().
3. Retrieve all parcels with delivery_status "Pending" using find().
4. Display only sender_name, receiver_name, and shipping_cost using projection.
5. Delete a parcel based on parcel_id.

Problem 10: Fitness Workout Logger

Problem Statement:

A fitness enthusiast wants to log daily workouts using MongoDB. Each workout contains workout_id, exercise_name, duration_minutes, calories_burned, date, and intensity (Low/Medium/High). The user needs basic logging.

Questions:

1. Create a workouts collection and insert one workout document.
2. Insert at least 5 workout records using insertMany().
3. Retrieve all workouts with intensity "High" using find().
4. Display only exercise_name, duration_minutes, and calories_burned using projection.
5. Delete a workout based on workout_id.

Problem 11: Subscription Management System

Problem Statement:

A streaming service wants to manage user subscriptions using MongoDB. Each subscription contains sub_id, user_name, plan_type (Basic/Standard/Premium), monthly_fee, start_date, renewal_date, and is_active (boolean). The admin needs to manage subscriptions.

Questions:

1. Insert 5 subscription documents using insertMany() with different plan types.
2. Find all subscriptions with is_active true AND plan_type "Premium".
3. Update the monthly_fee by increasing it by ₹100 for all Basic plan subscriptions.
4. Delete all subscriptions where is_active is false AND renewal_date is before "2025-01-01".

5. Find all subscriptions sorted by monthly_fee in descending order, displaying only user_name, plan_type, and monthly_fee.

Problem 12: Employee Attendance Tracker

Problem Statement:

A company wants to track employee attendance using MongoDB. Each attendance record contains record_id, emp_name, department, date, status (Present/Absent/Leave), and check_in_time. The HR team needs to manage attendance data.

Questions:

1. Insert 5 attendance records using `insertMany()` with different statuses.
2. Find all records where status is "Absent" AND department is "IT".
3. Update the status to "Present" for all records where check_in_time is not null.
4. Delete all records where date is before "2024-01-01" AND status is "Leave".
5. Find all records with status "Present" sorted by date in ascending order.

Problem 13: Online Polling System

Problem Statement:

An online polling platform wants to manage polls and votes using MongoDB. Each poll document contains poll_id, question, options (array), votes (array of vote counts), created_date, end_date, and is_active (boolean). The platform needs to manage polls.

Questions:

1. Insert 5 poll documents using `insertMany()` with different options arrays.
2. Find all polls where is_active is true AND end_date is after today's date.
3. Update the is_active to false for all polls where end_date is before today's date.
4. Delete all polls where created_date is before "2023-01-01" AND is_active is false.
5. Find all polls where the options array length is greater than 3 using `$size` operator.

Problem 14: Vehicle Service Center System

Problem Statement:

A vehicle service center wants to manage service records using MongoDB. Each service record contains service_id, customer_name, vehicle_number, service_type (Oil Change/Repair/Inspection), service_date, cost, and status (Scheduled/InProgress/Completed). The center needs to manage service appointments.

Questions:

1. Insert 5 service records using insertMany() with different service types.
2. Find all records with status "InProgress" AND service_date is before "2026-08-01".
3. Update the status to "Completed" for all records where service_date is before "2025-01-01".
4. Delete all records with status "Completed" AND cost is less than ₹500.
5. Find all records sorted by cost in descending order, displaying only customer_name, service_type, and cost.

Problem 15: Book Reading Challenge Tracker

Problem Statement:

A book club wants to track members' reading progress using MongoDB. Each reading record contains record_id, member_name, book_title, genre, pages_read, total_pages, start_date, completion_date, and is_completed (boolean). The club needs to manage reading challenges.

Questions:

1. Insert 5 reading records using insertMany() with different genres.
2. Find all records where is_completed is false AND pages_read is greater than 100.
3. Update the is_completed to true for all records where completion_date is not null.
4. Delete all records where start_date is before "2023-01-01" AND is_completed is false.
5. Find all records where genre is "Fiction" AND is_completed is true.

Problem 16: Pharmacy Prescription Tracker

Problem Statement:

A pharmacy wants to track customer prescriptions using MongoDB. Each prescription contains prescription_id, patient_name, doctor_name, medicine_name, dosage, issue_date, expiry_date, and status (Active/Expired/Fulfilled). The pharmacist needs to manage prescriptions.

Questions:

1. Insert 5 prescription documents using insertMany() with different statuses.
2. Find all prescriptions with status "Active" AND expiry_date is after today's date.
3. Update the status to "Expired" for all prescriptions where expiry_date is before today's date.
4. Delete all prescriptions with status "Fulfilled" AND issue_date before "2024-01-01".
5. Find all prescriptions sorted by issue_date in descending order, displaying only patient_name, medicine_name, and status.

Problem 17: Cab Ride Booking System

Problem Statement:

A cab service company wants to manage ride bookings using MongoDB. Each ride document contains ride_id, rider_name, driver_name, pickup_location, drop_location, ride_date, distance_km, fare, and ride_status (Requested/Accepted/Completed/Cancelled). The operations team needs to manage rides.

Questions:

1. Insert 5 ride documents using insertMany() with different ride statuses.
2. Find all rides with ride_status "Requested" AND ride_date is today.
3. Update the ride_status to "Completed" for all rides where drop_location is not null.
4. Delete all rides with ride_status "Cancelled" AND ride_date before "2024-06-01".
5. Find all rides where distance_km is greater than 10 AND fare is greater than ₹300.

Problem 18: Gym Equipment Maintenance Tracker

Problem Statement:

A gym wants to track equipment maintenance using MongoDB. Each equipment document contains equip_id, equip_name, category (Cardio/Strength), purchase_date, last_maintenance_date, next_maintenance_date, and condition (Good/NeedsService/OutOfOrder). The gym staff needs to manage maintenance schedules.

Questions:

1. Insert 5 equipment documents using insertMany() with different conditions.
2. Find all equipment where condition is "NeedsService" OR "OutOfOrder".
3. Update the condition to "Good" for all equipment where last_maintenance_date is after "2025-01-01".
4. Delete all equipment where purchase_date is before "2015-01-01" AND condition is "OutOfOrder".
5. Find all equipment sorted by next_maintenance_date in ascending order.

Problem 19: Online Course Review System

Problem Statement:

An e-learning platform wants to manage course reviews using MongoDB. Each review document contains review_id, course_name, student_name, rating (1-5), review_text, review_date, likes_count, and is_verified_purchase (boolean). The platform needs to manage reviews.

Questions:

1. Insert 5 review documents using insertMany() with different ratings.
2. Find all reviews with rating greater than 4 AND likes_count greater than 10.
3. Update the is_verified_purchase to true for all reviews where course_name contains "MongoDB".
4. Delete all reviews where review_date is before "2024-01-01" AND rating is 1.
5. Find all reviews sorted by rating in descending order, displaying only course_name, student_name, and rating.

Problem 20: Hospital Bed Availability System

Problem Statement:

A hospital wants to track bed availability using MongoDB. Each bed document contains bed_id, ward_name, bed_type (General/ICU/Private), is_occupied (boolean), patient_name (if occupied), admission_date, and expected_discharge_date. The hospital admin needs to manage bed allocation.

Questions:

1. Insert 5 bed documents using insertMany() with different bed types.
2. Find all beds where is_occupied is false AND ward_name is "ICU".
3. Update the is_occupied to true and set patient_name for beds where admission_date is today.
4. Delete all beds where expected_discharge_date is before today's date AND is_occupied is true.
5. Find all beds sorted by bed_type, displaying only ward_name, bed_type, and is_occupied.

Problem 21: E-Commerce Wishlist System

Problem Statement:

An e-commerce platform wants to manage customer wishlists using MongoDB. Each wishlist document contains wishlist_id, customer_name, product_name, product_category, price, added_date, and notify_when_available (boolean). The platform needs to manage wishlists.

Questions:

1. Insert 5 wishlist documents using insertMany() with different product categories.
2. Find all wishlist items where price is less than ₹2000 AND notify_when_available is true.
3. Update the price by applying a 20% discount for all products in "Electronics" category.
4. Delete all wishlist items where added_date is before "2024-01-01".
5. Find all wishlist items grouped by customer_name and count how many items each customer has.

Problem 22: Travel Itinerary Planner

Problem Statement:

A travel agency wants to manage customer itineraries using MongoDB. Each itinerary document contains itinerary_id, customer_name, destination, trip_start_date, trip_end_date, budget, activities (array), hotel_name, and booking_status (Confirmed/Pending/Cancelled). The agency needs to manage trip plans.

Questions:

1. Insert 5 itinerary documents using insertMany() with different destinations.
2. Find all itineraries where booking_status is "Confirmed" AND budget is greater than ₹50,000.
3. Update the booking_status to "Cancelled" for all itineraries where trip_start_date is before today's date.
4. Delete all itineraries with booking_status "Cancelled" AND trip_end_date before "2024-01-01".
5. Find all itineraries where the activities array contains "Trekking" using array query operator.

Problem 23: Warehouse Inventory System

Problem Statement:

A warehouse wants to manage inventory using MongoDB. Each product document contains product_id, product_name, category, quantity_in_stock, reorder_level, supplier_name, last_restock_date, and storage_location. The warehouse manager needs inventory management.

Questions:

1. Insert 5 product documents using insertMany() with different categories.
2. Find all products where quantity_in_stock is less than reorder_level.
3. Update the quantity_in_stock by adding 100 to all products from supplier "ABC Corp".
4. Delete all products where last_restock_date is before "2022-01-01" AND quantity_in_stock is 0.

5. Create an index on category and quantity_in_stock to optimize inventory queries.

Problem 24: Music Playlist Manager

Problem Statement:

A music streaming app wants to manage user playlists using MongoDB. Each playlist document contains playlist_id, user_name, playlist_name, songs (array of song objects with title and artist), created_date, total_duration_minutes, is_public (boolean), and play_count. The app needs to manage playlists.

Questions:

1. Insert 5 playlist documents using insertMany() with different songs arrays.
2. Find all playlists where is_public is true AND play_count is greater than 100.
3. Update the play_count by incrementing it by 1 for a specific playlist_id.
4. Delete all playlists where created_date is before "2023-01-01" AND is_public is false.
5. Use aggregation to find the average total_duration_minutes per user.

Problem 25: Banking Loan Application System

Problem Statement:

A bank wants to manage loan applications using MongoDB. Each loan document contains loan_id, applicant_name, loan_type (Home/Car/Personal), loan_amount, interest_rate, tenure_months, application_date, approval_status (Pending/Approved/Rejected), and credit_score. The bank needs to analyze loan applications.

Questions:

1. Insert 5 loan documents using insertMany() with different loan types.
2. Find all loans where approval_status is "Pending" AND credit_score is greater than 700.
3. Update the approval_status to "Approved" for all loans where credit_score is greater than 750.
4. Delete all loans where application_date is before "2023-01-01" AND approval_status is "Rejected".

5. Use aggregation to calculate the average loan_amount per loan_type for approved loans only.

Problem 26: Food Delivery Rider Tracking System

Problem Statement:

A food delivery company wants to track delivery riders using MongoDB. Each rider document contains rider_id, rider_name, city, vehicle_type (Bike/Scooter), total_deliveries, average_rating, current_status (Available/Busy/Offline), earnings_today, and delivery_history (array of order_ids). The operations team needs rider management.

Questions:

1. Insert 5 rider documents using `insertMany()` with different current_status values.
2. Find all riders where current_status is "Available" AND average_rating is greater than 4.5.
3. Update the earnings_today by adding ₹100 to all riders with total_deliveries greater than 20.
4. Delete all riders where current_status is "Offline" AND total_deliveries is 0.
5. Create a multikey index on the `delivery_history` array field.

Problem 27: Hotel Guest Feedback System

Problem Statement:

A hotel chain wants to analyze guest feedback using MongoDB. Each feedback document contains feedback_id, guest_name, hotel_name, room_number, stay_dates, cleanliness_rating (1-5), service_rating (1-5), overall_rating (1-5), comments, feedback_date, and would_recommend (boolean). The hotel management needs feedback analysis.

Questions:

1. Insert 5 feedback documents using `insertMany()` with different ratings.
2. Find all feedback where overall_rating is less than 3 AND would_recommend is false.

3. Update the would_recommend to true for all feedback where overall_rating is greater than 4.
4. Delete all feedback where feedback_date is before "2023-01-01" AND overall_rating is 1.
5. Use aggregation to calculate the average cleanliness_rating and service_rating per hotel_name.

Problem 28: Event Ticket Booking System

Problem Statement:

An event management company wants to manage ticket bookings using MongoDB. Each booking document contains booking_id, event_name, event_date, venue, customer_name, ticket_type (VIP/General/Student), number_of_tickets, total_price, booking_date, and payment_status (Paid/Pending/Refunded). The company needs booking management.

Questions:

1. Insert 5 booking documents using insertMany() with different ticket types.
2. Find all bookings where event_date is after "2026-10-01" AND payment_status is "Paid".
3. Update the payment_status to "Refunded" for all bookings where event_date is before today's date AND payment_status is "Paid".
4. Delete all bookings with payment_status "Pending" AND booking_date before "2024-01-01".
5. Use aggregation to calculate the total number_of_tickets sold per event_name.

Problem 29: Social Media Follower Analytics

Problem Statement:

A social media influencer wants to analyze follower growth using MongoDB. Each follower document contains follower_id, follower_name, followed_since, engagement_score (1-100), is_active (boolean), location, interests (array), and last_interaction_date. The influencer needs follower analytics.

Questions:

1. Insert 5 follower documents using insertMany() with different interests arrays.
2. Find all followers where engagement_score is greater than 80 AND is_active is true.
3. Update the is_active to false for all followers where last_interaction_date is before "2025-01-01".
4. Delete all followers where followed_since is before "2022-01-01" AND engagement_score is less than 20.
5. Use aggregation with $unwind to find the most common interest among followers.

Problem 30: Manufacturing Defect Tracking System

Problem Statement:

A manufacturing company wants to track product defects using MongoDB. Each defect record contains defect_id, product_name, batch_number, defect_type (Cosmetic/Functional/Safety), detection_date, severity (Low/Medium/High), quantity_affected, root_cause, status (Open/Investigating/Resolved), and resolution_date. The quality team needs defect analysis.

Questions:

1. Insert 5 defect records using insertMany() with different severity levels.
2. Find all defects where severity is "High" AND status is "Open".
3. Update the status to "Resolved" and set resolution_date to today for all defects where root_cause is identified.
4. Delete all defects where detection_date is before "2022-01-01" AND status is "Resolved".
5. Use aggregation to calculate the total quantity_affected per defect_type, sorted from highest to lowest.